

Question

1

2

3

4

5

6

7

8

9

10

10

list ใน python นั้นเป็นข้อมูลชนิด mutable ซึ่งสามารถเปลี่ยนแปลงค่าได้
โดยจากโค้ดข้างล่าง จงเติมค่าให้ผลลัพธ์ออกมาถูกต้อง

```
>>> list1 = ["tree", "banana", "apple", "pie"]
```

```
>>> list1[1] = '5555'
```

```
>>> print(list1)
```

```
['tree', '5555', 'apple', 'pie']
```


Question

1

2

3

4

5

6

7

8



10

9

จงเติมช่องว่างให้ถูกต้อง

```
>>>my_set = 
```

```
>>> my_set.add(1)
```

```
>>> my_set.add(2)
```

```
>>> my_set.add(2)
```

```
>>> my_set.add(3)
```

```
>>> print(my_set)
```

```
{1, 2, 3}
```


Description

List Dict Tuple Set 1

Question

1

2

3

4

5

6

7

8

9

10

8

```
>>> numjp2 = {'0': "zero", '6': "roku", '7': "nana", '8': "hachi", '9': "kyuu"}
```

```
>>> print(numjp2.get(1, 'unknown'))
```

```
unknown
```

```
>>> print(numjp2)
```

```
{'0': 'zero', '6': 'roku', '7': 'nana', '8': 'hachi', '9': 'kyuu'}
```

ให้หาวิธีดึงค่าออกมาจากใน dict ของ numjp2 เพื่อให้แสดงผลออกมาเป็นคำว่า 'un

Question

1

2

3

4

5

6

7

8

9

10

8

การเพิ่มค่าเข้าไปในลิสไม่ได้มีเพียง append แต่ยังมีวิธีอื่นซึ่งแตกต่างกันออกไป
จงเติมโค้ดต่อไปนี้ให้ถูกต้อง

```
>>> list_a = [1, 2]
```

```
>>> list_a.extend([3, 4, 5])
```

```
>>> print(list_a)
```

```
[1, 2, 3, 4, 5]
```


Question

1

2

3

4

5

6

7

8

9

10

```
def fun(a, b):  
    return a+b
```

ถ้า `t = (1, 2)`

จะเรียก function fun โดยส่งค่า t เข้าไปเป็น argument เช่น

`x = fun(t)` จะเกิด error ขึ้น

ถามว่าจะต้องเรียก function fun นี้อย่างไร จึงไม่เกิด error แล้ว ret

`x = fun(*t`

หมายเหตุ ห้ามเรียกผ่าน index และห้ามใช้ method ของ tuple

Question

1

2

3

4

5

6

7

8

9

10

7

จงเติมโค้ดข้างล่างเพื่อให้โค้ดสมบูรณ์

```
>>> list_a = [1, 2, 3]
```

```
>>> list_b = list_a[0:3]
```

```
>>> print(list_b)
```

```
[1, 2, 3]
```

```
>>> list_b.pop()
```

```
>>> print(list_b)
```

```
[1, 2]
```

```
>>> print(list_a)
```

```
[1, 2, 3]
```


Description

List Dict Tuple Set 1

Question

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

8

```
>>> numjp2 = {'0': "zero", '6': "roku", '7': "nana", '8': "hachi", '9': "kyuu"}
```

```
>>> print(numjp2.get(1, 'unknown'))
```

unknown

```
>>> print(numjp2)
```

```
{'0': 'zero', '6': 'roku', '7': 'nana', '8': 'hachi', '9': 'kyuu'}
```

ให้หาวิธีดึงค่าออกมาจากใน dict ของ numjp2 เพื่อให้แสดงผลออกมาเป็นคำว่า 'un

Description

List Dict Tuple Set 1

Question

1

2

3

4

5

6

7

8

9

10

7

```
>>> numjp = {1:"ichi", 2:"ni", 3:"san", 4:"yon", 5:"go"}  
>>> print(numjp[3])  
san
```

ให้หาวิธีดึงค่าออกมาจากใน dict ของ numjp เพื่อให้แสดงผลออกมาเป็นคำว่า


```
>>> b = 'banana'
```

```
>>> a == b
```

```
True
```

```
>>> a is b
```

```
True
```

```
>>> a = [1,2,3]
```

```
>>> b = [1,2,3]
```

```
>>> a == b
```

```
True
```

```
>>> a is b
```

```
False
```

```
>>> a = b
```

```
>>> a == b
```

```
True
```

```
>>> a is b
```

```
True
```

```
>>> b[0] = 10
```

```
>>> b
```

```
[10, 2, 3]
```

```
>>> a
```

```
[10, 2, 3]
```

```
>>> a = b[:]
```

```
>>> a == b
```

```
True
```

```
>>> a is b
```

```
False
```


Description

List Dict Tuple Set 1

Question

1

2



4

5

6

7

8

9

10

3

```
>>> x = 5
```

```
>>> y = 10
```

```
>>> x, y = y, x
```

```
>>> print(x)
```

```
10
```

```
>>> print(y)
```

```
5
```

จงเติม statement ลงในช่องว่าง

List Dict Tuple Set 1

Question

1

2

3

4

5

6

7

8

9

10

2

```
>>> numjp = {1:"ichi", 2:"ni", 3:"san", 4:"yon", 5:"go"}
>>> jpcount = list(numjp.values())
>>> print(jpcount)
['ichi', 'ni', 'san', 'yon', 'go']
```

จงเติมค่าในช่องว่างเพื่อให้ตัวแปร jpcount สามารถแสดงผลได้เหมือนกับด้านล่าง

Hint: ดูจุดกับวงเล็บให้ดีๆ นะ หลัง numjp มีจุดแถมมาให้อยู่แล้ว

Description

List Dict Tuple Set 1

Question

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10

4

is a built-in function that takes two or more sequences and returns a sequence of tuples, one for each sequence.

Dictionaries have a method called that returns a sequence of (key, value) tuples.

Question

1

2

3

4

5

6

7

8

1

เติมผลลัพธ์การเปรียบเทียบลงในช่องว่าง

>>> (10, 'hello') > (9, 'moon')

True

>>> ('moon', 10) > ('world', 20)

False

>>> ('10', 'world') > ('2', 'world')

False

>>> (10, 11, 12, 1) > (10, 11, 12, 2)

False